Data Analysis

Classification

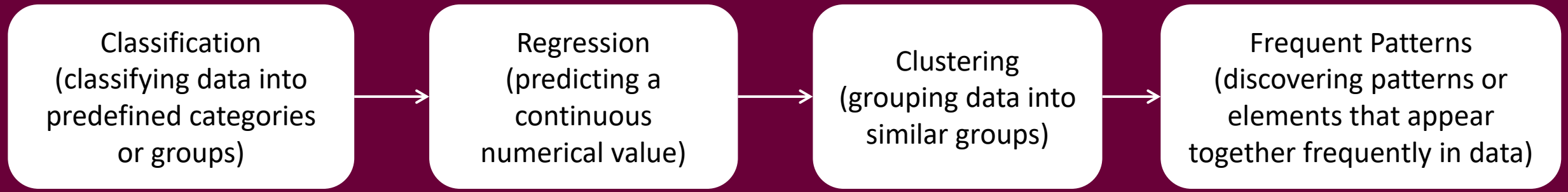
Section 3

Eng. Asmaa Fathy

What is Data Analysis ?

- Data analysis is the process of examining, organizing, and interpreting data to extract useful information that helps in understanding a particular problem or making a better decision..

- Tasks of Data Analysis:



Classification

- NEAREST NEIGHBOR CLASSIFIERS
- DECISION TREE CLASSIFIERS
- SUPPORT VECTOR MACHINE CLASSIFIERS
- NAIVE BAYES CLASSIFIERS
- LOGISTIC REGRESSION CLASSIFIERS

NEAREST NEIGHBOR CLASSIFIERS

- Nearest Neighbor Classifiers are algorithms in Machine Learning and Pattern Recognition that rely on a simple idea: Similar points in the data often belong to the same class.
- So, if we have a new point, we look at the closest points to it in the data and, based on that, determine its class.
- **There are two essential Nearest Neighbor Classifiers: K-Nearest Neighbors (KNN) and Radius Neighbors (RNN)**
- **Nearest Neighbor Example: Imagine we have data for students:**

Student	Study Hours	Result
A	2	Fail
B	3	Fail
C	7	Pass
D	8	Pass

- **Now a new student comes in: Study Hours = 6**
- **We'll look at the students closest to him in terms of study hours.**

Closest to him:

7 → Pass

8 → Pass

- **Therefore, we expect the student to have passed.**
- **The idea here is that the new point takes the classification of its nearest neighbors**

K-Nearest Neighbors (KNN)

- KNN is a Supervised Machine Learning algorithm used for Classification and Regression.
- It relies on the principle that data with similar characteristics often belong to the same class.
- How it Works? When a new point is reached:

1. We calculate the distance between it and all points in the data.
2. We select the nearest K points.
3. The most frequently occurring class among them is the result.

- **Example of K-Nearest Neighbors (KNN): Let's assume: K = 3 and we have the 3 nearest neighbors to the new point:**

Neighbor	Class
1	Red
2	Red
3	Blue

We calculate the majority:
Red = 2
Blue = 1
Therefore, the final classification = Red
This means the decision is based on the majority of neighbors

Radius Neighbors (RNN)

- Radius Neighbors is an extension of the KNN algorithm. However, instead of choosing a fixed number of neighbors (K), we select all points within a specified distance (Radius) from the new point.
- How it Works?

1. We determine the value of the radius.
2. We search for all points within this distance.
3. We use them to determine the classification.

- **Example of Radius Neighbors (RNN): Instead of saying the 3 nearest points, we say we look for all points within a certain distance. For example: Radius = 2, if the points within the distance are:**

Point	Class
A	Green
B	Green
C	Yellow

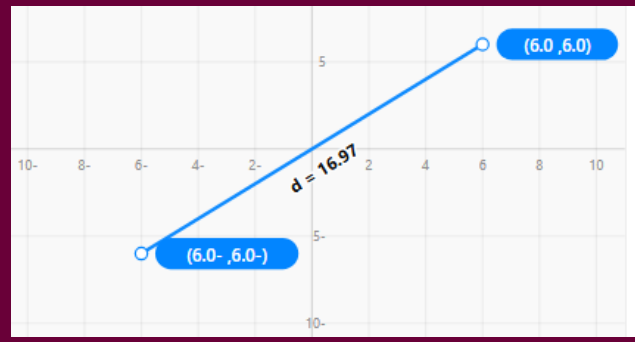
The classification = Green because it's the most common.
But sometimes it's possible: Find 5 points, or only 2 points
Depending on the specified distance.

Difference Between KNN and RNN:

KNN	RNN
Depends on the number of neighbors (K)	Depends on a specific distance,
Always chooses the same number	may choose a different number of points

What is Radius in RNN?

Radius is simply distance. This means we draw a circle around the new point, and all points inside the circle are considered neighbors. Points outside the circle are not used.
How is distance calculated? In most cases, it is calculated using Euclidean Distance. Its concept is similar to the distance between two points on a plane.



$$d = sqrt((x1 - x2)^2 + (y1 - y2)^2)$$

This means: We calculate the difference between the values, We square the difference, We add them, and We take the square root

Let's assume we have two points:

New Point	
Sepal length	Sepal width
6	3

Point in data	
Sepal length	Sepal width
5	3

We calculate the distance:
(6 - 5)^2 = 1
(3 - 3)^2 = 0
Distance = √(1 + 0) = 1

- Now we apply the Radius **If: radius = 1** → . This point is considered a Neighbor.
- But **if the distance is: 1.5** It will not be included in the calculation because it is outside the circle.
- **The larger the radius → the larger the circle → more neighbors are included.**

Iris Dataset.

- It is one of the most popular datasets in Machine Learning, and it's used to classify flower types.
- **Number of samples: 150 flowers**
- In the iris flower, two important parts were measured in the data:

1. **Sepals** are the green leaves surrounding the flower, whose primary function is to protect it before it opens.
2. **Petals** are the colored leaves inside the flower, whose function is to attract insects for pollination.

- We use these measurements to identify the flower species using algorithms such as KNN or RNN because the shape and size of these parts vary between flower species. For example, one type of flower may have long petals, while another type has short petals.

The data contains 3 types of Iris flowers (species):

0	1	2
Setosa	Versicolor	Virginica

Each flower in the data has 4 features measured:

Feature	Type
Sepal length	Numeric
Sepal width	Numeric
Petal length	Numeric
Petal width	Numeric

Iris Binary Classification Using KNN

```python
# Setup Environment
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt # For data plotting
import seaborn as sns # For data plotting
from sklearn import datasets # Machine Learning library
```

```python
# Download Data
# We load the ready-made Iris dataset from Scikit-Learn
iris = datasets.load_iris()
```

```python
# Converting Data to a DataFrame
df = pd.DataFrame({
'Sepal length': iris.data[:,0],
'Sepal width': iris.data[:,1],
'Petal length': iris.data[:,2],
'Petal width': iris.data[:,3],
'Species': iris.target
})

df.head()
```

	Sepal length	Sepal width	Petal length	Petal width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

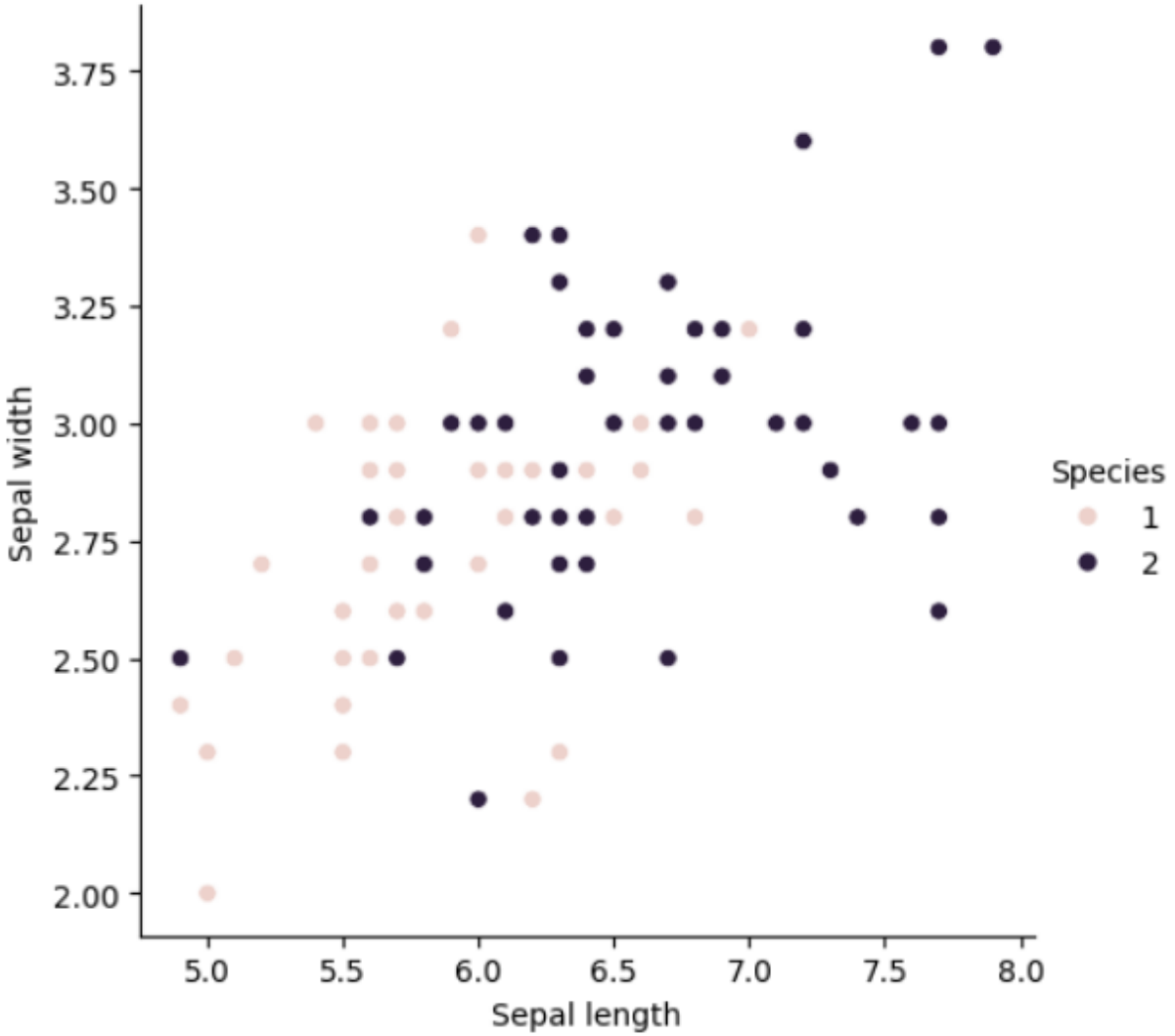
```python
# Deleting one flower type

df = df[df['Species'] != 0]

# The Iris dataset contains 3 flower types: 0 → Setosa | 1 → Versicolor | 2 → Virginica
# We deleted type 0 to make the problem a binary classification (only two types).
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 100 entries, 50 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Sepal length  100 non-null    float64
 1   Sepal width   100 non-null    float64
 2   Petal length  100 non-null    float64
 3   Petal width   100 non-null    float64
 4   Species       100 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 4.7 KB
```

```
# Visualization (Data Plotting)

sns.relplot(data=df, x='Sepal length', y='Sepal width', hue='Species')

# We have drawn a Scatter Plot to see if the types are separated in the data or not:
# X → Sepal length, Y → Sepal width, The colors represent the flower type.
```

```
<seaborn.axisgrid.FacetGrid at 0x23b8013b380>
```



```python
# Data Splitting

from sklearn.model_selection import train_test_split
X = df[df.columns[:2]] # Features
y = df[df.columns[-1]] # Species

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.50)
# We split the data into:
# Training | 50% → To train the model
# Testing | 50% → To evaluate it

X_train[:5]
```

	Sepal length	Sepal width
95	5.7	3.0
103	6.3	2.9
63	6.1	2.9
79	5.7	2.6
54	6.5	2.8

```python
# Scaling the Data: standardizing the scale of the data
# Because: Some properties may have large values, some may be small, and KNN relies on distance.

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train) # Calculates mean and std from the training data
X_train = scaler.transform(X_train) # Transforms the training data
X_test = scaler.transform(X_test) # Transforms the test data with the same values

X_train[:5]
```

```
array([[-0.72624384,  0.51193302],
       [ 0.14177669,  0.21771864],
       [-0.14756349,  0.21771864],
       [-0.72624384, -0.66492449],
       [ 0.43111686, -0.07649574]])
```

- We don't fit test data to avoid data leakage.
- **Data leakage** means the model has seen the test data while learning, resulting in inaccurate results.
- Imagine: You have student data: study hours → exam scores. Your goal is to predict the next student's score. If you use the same student's score in the training data during model training, this is data leakage.
- The model already knows the answer, giving high but unrealistic accuracy on new data.

```python
# Training the KNN Model

from sklearn.neighbors import KNeighborsClassifier
k = 1  #  This means the classification is based on only the nearest neighbor
classifier = KNeighborsClassifier(n_neighbors=k)
classifier.fit(X_train, y_train)
```

```python
# Predicting the Results
# The model attempts to predict the type of flower for the new data.

y_pred = classifier.predict(X_test)
```

```python
# Model Evaluation
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# We used 3 metrics:

# 1- Confusion Matrix: This shows how many cases were classified correctly and how many were classified incorrectly
result = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(result)

# 2- Classification Report: This gives Precision, Recall, and F1 score
result1 = classification_report(y_test, y_pred)
print("Classification Report:",)
print (result1)

# 3- Accuracy:
result2 = accuracy_score(y_test,y_pred)
print("Accuracy:",result2)
# This means the model is correct in 70% of cases.
```

```
Confusion Matrix:
[[16  5]
 [10 19]]
Classification Report:
              precision    recall  f1-score   support

           1       0.62      0.76      0.68        21
           2       0.79      0.66      0.72        29

    accuracy                           0.70        50
   macro avg       0.70      0.71      0.70        50
weighted avg       0.72      0.70      0.70        50

Accuracy: 0.7
```

```python
# Finding the Best K: This function tries different values for K.
# For example: K = 1 , K = 5 , K = 11, K = 16
# We calculate the accuracy for each value.

def knn_tuning(k):
    classifier = KNeighborsClassifier(n_neighbors=k)
    classifier.fit(X_train,y_train)
    y_pred = classifier.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    return accuracy
```

```python
knn_tuning(1)
```

```
0.6
```

```python
knn_tuning(5)
```

```
0.62
```

```python
knn_results = pd.DataFrame({'K':np.arange(1,len(X_train),5)}) # (1, 50 , 5)
knn_results['K']
```

```
0     1
1     6
2    11
3    16
4    21
5    26
6    31
7    36
8    41
9    46
Name: K, dtype: int64
```

```python
knn_results['Accuracy']= knn_results['K'].apply(knn_tuning)
knn_results['Accuracy']
```

```
0    0.60
1    0.52
2    0.58
3    0.58
4    0.70
5    0.62
6    0.60
7    0.64
8    0.58
9    0.42
Name: Accuracy, dtype: float64
```

```python
knn_results
```

	K	Accuracy
0	1	0.60
1	6	0.52
2	11	0.58
3	16	0.58
4	21	0.70
5	26	0.62
6	31	0.60
7	36	0.64
8	41	0.58
9	46	0.42

Optimizing Weights

- Weight: determines the influence of each neighbor on the final decision.
- **Two types of Weights:**

1. Uniform: Each neighbor has the **same influence** regardless of distance. Whether a neighbor is near or far → is considered equally influential.

Neighbor	Class	Influence
1	Red	1
2	Red	1
3	Blue	1
Output: Majority → Red		

2. Distance: **Closer neighbors have a greater impact**. Further, neighbors have a lesser impact.

Neighbor	Class	Distance	Weight = 1/Distance
1	Red	0.5	1/0.5 = 2
2	Red	1.0	1/1.0 = 1
3	Blue	1.5	1/1.5 ≈ 0.66
Output: Majority by Weighted Points → Red			

```
def knn_tuning_uniform(k):
    classifier = KNeighborsClassifier(n_neighbors = k, weights= 'uniform')
    classifier.fit(X_train, y_train)
    y_pred = classifier.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    return accuracy
```

```
def knn_tuning_distance(k):
    classifier = KNeighborsClassifier(n_neighbors = k, weights= 'distance')
    classifier.fit(X_train, y_train)
    y_pred = classifier.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    return accuracy
```

```
knn_results['Uniform'] = knn_results['K'].apply(knn_tuning_uniform)
knn_results['Distance'] = knn_results['K'].apply(knn_tuning_distance)
knn_results
```

	K	Accuracy	Uniform	Distance
0	1	0.60	0.60	0.60
1	6	0.52	0.52	0.62
2	11	0.58	0.58	0.58
3	16	0.58	0.58	0.62
4	21	0.70	0.70	0.64
5	26	0.62	0.62	0.62
6	31	0.60	0.60	0.60
7	36	0.64	0.64	0.60
8	41	0.58	0.58	0.60
9	46	0.42	0.42	0.56

Iris Binary Classification Using RNN

```python
# Creating the Model
r = 1 # radius --> This means we will search for **all neighbors within a distance of 1.

from sklearn.neighbors import RadiusNeighborsClassifier
classifier = RadiusNeighborsClassifier(radius = r)

# Train Model
classifier.fit(X_train, y_train)
```

```python
# Predicting the Outcome
y_pred = classifier.predict(X_test)
```

```python
# Model Evaluation
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

result = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(result)

result1 = classification_report(y_test, y_pred)
print("Classification Report:",)
print (result1)

result2 = accuracy_score(y_test,y_pred)
print("Accuracy:",result2)
```

```
Confusion Matrix:
[[18  6]
 [12 14]]
Classification Report:
              precision    recall  f1-score   support

           1       0.60      0.75      0.67        24
           2       0.70      0.54      0.61        26

    accuracy                           0.64        50
   macro avg       0.65      0.64      0.64        50
weighted avg       0.65      0.64      0.64        50

Accuracy: 0.64
```

```python
# Finding the Best Radius
# We try different values for radius and calculate accuracy for each value.
def rnn_tuning(r):
    classifier = RadiusNeighborsClassifier(radius = r)
    classifier.fit(X_train, y_train)
    y_pred = classifier.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    return accuracy
```

```python
rnn_tuning(1)
```

```
0.64
```

```python
rnn_tuning(5)
```

```
0.5
```

```python
rnn_results=pd.DataFrame({'R':np.arange(1,10,0.5)})
rnn_results['R']
```

```
0     1.0
1     1.5
2     2.0
3     2.5
4     3.0
5     3.5
6     4.0
7     4.5
8     5.0
9     5.5
10    6.0
11    6.5
12    7.0
13    7.5
14    8.0
15    8.5
16    9.0
17    9.5
Name: R, dtype: float64
```

```python
rnn_results['Accuracy']=rnn_results['R'].apply(rnn_tuning)
rnn_results['Accuracy']
```

```
0     0.64
1     0.66
2     0.68
3     0.68
4     0.68
5     0.60
6     0.52
7     0.50
8     0.50
9     0.50
10    0.48
11    0.48
12    0.48
13    0.48
14    0.48
15    0.48
16    0.48
17    0.48
Name: Accuracy, dtype: float64
```

rnn_results

	R	Accuracy
0	1.0	0.64
1	1.5	0.66
2	2.0	0.68
3	2.5	0.68
4	3.0	0.68
5	3.5	0.60
6	4.0	0.52
7	4.5	0.50
8	5.0	0.50
9	5.5	0.50
10	6.0	0.48
11	6.5	0.48
12	7.0	0.48
13	7.5	0.48
14	8.0	0.48
15	8.5	0.48
16	9.0	0.48
17	9.5	0.48

Optimizing Weights

```
def rnn_tuning_uniform(r):
    classifier=RadiusNeighborsClassifier(radius=r,weights='uniform')
    classifier.fit(X_train,y_train)
    y_pred=classifier.predict(X_test)
    accuracy=accuracy_score(y_test,y_pred)
    return accuracy
```

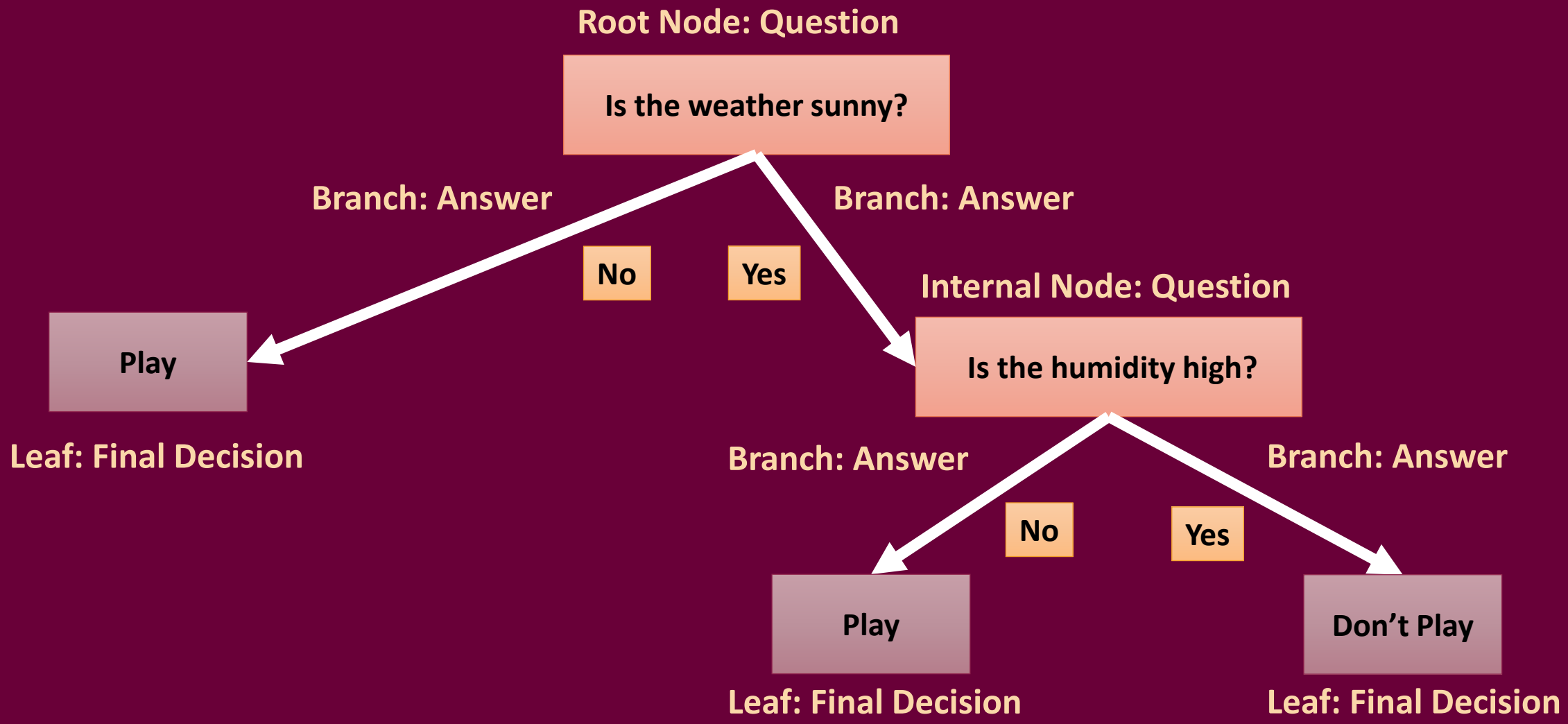
```
def rnn_tuning_distance(k):
    classifier=RadiusNeighborsClassifier(radius=k,weights='distance')
    classifier.fit(X_train,y_train)
    y_pred=classifier.predict(X_test)
    accuracy=accuracy_score(y_test,y_pred)
    return accuracy
```

```
rnn_results['Uniform']=rnn_results['R'].apply(rnn_tuning_uniform)
rnn_results['Distance']=rnn_results['R'].apply(rnn_tuning_distance)
rnn_results
```

	R	Accuracy	Uniform	Distance
0	1.0	0.64	0.64	0.56
1	1.5	0.66	0.66	0.56
2	2.0	0.68	0.68	0.58
3	2.5	0.68	0.68	0.58
4	3.0	0.68	0.68	0.60
5	3.5	0.60	0.60	0.58
6	4.0	0.52	0.52	0.60
7	4.5	0.50	0.50	0.60
8	5.0	0.50	0.50	0.60
9	5.5	0.50	0.50	0.60
10	6.0	0.48	0.48	0.60
11	6.5	0.48	0.48	0.60
12	7.0	0.48	0.48	0.60
13	7.5	0.48	0.48	0.60
14	8.0	0.48	0.48	0.60
15	8.5	0.48	0.48	0.60
16	9.0	0.48	0.48	0.60
17	9.5	0.48	0.48	0.60

DECISION TREE CLASSIFIERS

- A decision tree is an algorithm in the field of Machine Learning used for classification and regression.
- It's a decision tree, like a series of questions.
- Each node in the tree represents a question, each branch represents an answer, and each leaf represents the final decision.



Decision Tree Induction: Attribute Selection and Splitting Measures

- When building a decision tree, the key step is to decide which attribute to split the data on at each node.
- **Attribute selection measures (ASMs)** are used to evaluate the "best" attribute for splitting, ensuring the tree is as efficient and accurate as possible.
- One commonly used measure is **Information Gain (IG).**
- **Entropy (H):** A measure of uncertainty in the dataset. If all values in the target variable are the same, entropy is 0 (low uncertainty).
- **Information Gain measures the reduction in uncertainty (or "entropy")** about the target variable after splitting the dataset on a particular attribute. Attributes with higher Information Gain are preferred because they provide the most information about the target variable.
- **Information Gain (IG):** The difference between the entropy of the dataset before and after splitting on an attribute.

Entropy Formula

$$H(Y) = -\sum_{i=1}^{m} p_i \log(p_i) \quad where\ p_i = P(Y = y_i)$$

If all the data is from the same class
→ entropy = 0
→ The data is very clear.

Play
Yes
Yes
Yes
Yes

If the data is mixed
→ entropy ≠ 0
→ entropy is high.

Play
Yes
Yes
No
No

Information Gain (IG) Formula

- The amount of uncertainty reduced after partitioning

Expected information (entropy) needed to classify a tuple in D:

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

Information needed (after using A to split D into v partitions) to classify D:

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- S_v: Subset of S where attribute A has value v.
- $H(S_v)$: Entropy of the subset.

Example Problem

- Scenario: You are building a decision tree to predict whether a customer will buy a product (Target Variable: Buy (Yes/No)) based on the following attributes:
    1. Age: Young, Middle-aged, Senior
    2. Income: Low, Medium, High
    3. Student: Yes, No
    4. Credit Rating: Fair, Excellent

Dataset

Age	Income	Student	Credit Rating	Buy
Young	High	No	Fair	No
Young	High	No	Excellent	No
Middle-aged	High	No	Fair	Yes
Senior	Medium	No	Fair	Yes
Senior	Low	Yes	Fair	Yes
Senior	Low	Yes	Excellent	No
Middle-aged	Low	Yes	Excellent	Yes
Young	Medium	No	Fair	No
Young	Low	Yes	Fair	Yes
Senior	Medium	Yes	Fair	Yes
Young	Medium	Yes	Excellent	Yes
Middle-aged	Medium	No	Excellent	Yes
Middle-aged	High	Yes	Fair	Yes
Senior	Medium	No	Excellent	No

1) Calculating the Entropy of the Entire Dataset

Number of Records = 14

Number of:

Buy = Yes → 9

Buy = No → 5

We use the Entropy formula:
p(Yes) = 9/14
p(No) = 5/14

$$H(Y) = -\sum_{i=1}^{m} p_i \log(p_i) \quad where\ p_i = P(Y = y_i)$$

$$H(S) = -\left(\frac{9}{14}\log_2\frac{9}{14} + \frac{5}{14}\log_2\frac{5}{14}\right)$$

$$H(S) = -\left(0.643 \times \log_2(0.643) + 0.357 \times \log_2(0.357)\right) = 0.940$$

2) Calculating the Information Gain for Each Feature

Expected information (entropy) needed to classify a tuple in D:

$$Info(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

Information needed (after using A to split D into v partitions) to classify D:

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

Information gained by branching on attribute A

$$Gain(A) = Info(D) - Info_A(D)$$

1. Age

Split the dataset by **Age** into subsets:

- **Young**: 5 records $(2Yes, 3No)$,

$$H(Young) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) = 0.971$$

- **Middle-aged**: 4 records $(4Yes, 0No)$, $H(Middle-aged) = 0.0$.
- **Senior**: 5 records $(3Yes, 2No)$,

$$H(Senior) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$$

Weighted Entropy for Age:

$$H(S_{Age}) = \frac{5}{14}\cdot 0.971 + \frac{4}{14}\cdot 0.0 + \frac{5}{14}\cdot 0.971 = 0.693$$

Information Gain for Age:

$$IG(S, \text{Age}) = H(S) - H(S_{Age}) = 0.940 - 0.693 = 0.247$$

2. Income

Split the dataset by **Income** into subsets:

- **Low**: 4 records $(3Yes, 1No)$,

$$H(Low) = -\left(\frac{3}{4}\log_2\frac{3}{4} + \frac{1}{4}\log_2\frac{1}{4}\right) = 0.811$$

- **Medium**: 6 records $(4Yes, 2No)$,

$$H(Medium) = -\left(\frac{4}{6}\log_2\frac{4}{6} + \frac{2}{6}\log_2\frac{2}{6}\right) = 0.918$$

- **High**: 4 records $(2Yes, 2No)$,

$$H(High) = -\left(\frac{2}{4}\log_2\frac{2}{4} + \frac{2}{4}\log_2\frac{2}{4}\right) = 1.0$$

Weighted Entropy for Income:

$$H(S_{Income}) = \frac{4}{14}\cdot 0.811 + \frac{6}{14}\cdot 0.918 + \frac{4}{14}\cdot 1.0 = 0.889$$

Information Gain for Income:

$$IG(S, \text{Income}) = H(S) - H(S_{Income}) = 0.940 - 0.889 = 0.051$$

3. Student

Split the dataset by **Student**:

- **Yes**: 6 records $(5Yes, 1No)$,

$$H(Yes) = -\left(\frac{5}{6}\log_2\frac{5}{6} + \frac{1}{6}\log_2\frac{1}{6}\right) = 0.650$$

- **No**: 8 records $(4Yes, 4No)$,

$$H(No) = -\left(\frac{4}{8}\log_2\frac{4}{8} + \frac{4}{8}\log_2\frac{4}{8}\right) = 1.0$$

Weighted Entropy for Student:

$$H(S_{Student}) = \frac{6}{14}\cdot 0.650 + \frac{8}{14}\cdot 1.0 = 0.836$$

Information Gain for Student:

$$IG(S, \text{Student}) = H(S) - H(S_{Student}) = 0.940 - 0.836 = 0.104$$

4. Credit Rating

Split the dataset by **Credit Rating**:

- **Fair**: 9 records $(6Yes, 3No)$,

$$H(Fair) = -\left(\frac{6}{9}\log_2\frac{6}{9} + \frac{3}{9}\log_2\frac{3}{9}\right) = 0.918$$

- **Excellent**: 5 records $(3Yes, 2No)$,

$$H(Excellent) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$$

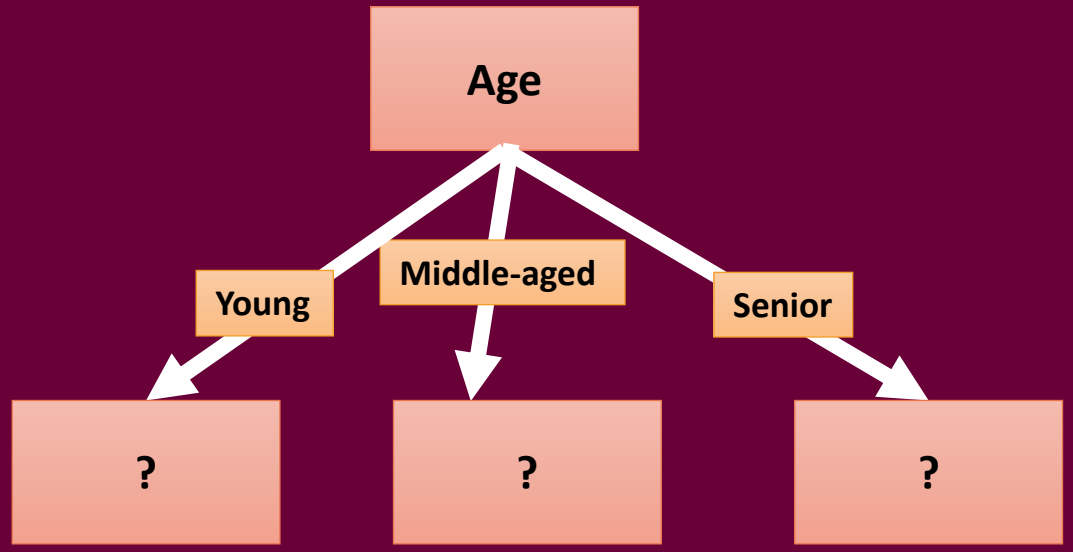
Weighted Entropy for Credit Rating:

$$H(S_{Credit}) = \frac{9}{14}\cdot 0.918 + \frac{5}{14}\cdot 0.971 = 0.936$$

Information Gain for Credit Rating:

$$IG(S, \text{Credit Rating}) = H(S) - H(S_{Credit}) = 0.940 - 0.936 = 0.004$$

3) Choose the Attribute with the Highest Information Gain



Based on the calculated Information Gains:

Age: 0.247	Income: 0.051	Student: 0.104	Credit Rating: 0.004

The attribute **Age** has the highest Information Gain and is chosen as the root node of the decision tree.

Young Branch

A	B	C	D	E
Age	Income	Student	Credit Rating	Buy
Young	High	No	Fair	No
Young	High	No	Excellent	No
Young	Medium	No	Fair	No
Young	Low	Yes	Fair	Yes
Young	Medium	Yes	Excellent	Yes

Number of Records = 5

Number of:

Buy = Yes → 2

Buy = No → 3

Calculating the Entropy

$$H(Young) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right)$$

$$H(Young) = -(0.4\,log2\,0.4 + 0.6\,log2\,0.6) \approx 0.971$$

1. Income

- **Split the dataset into subsets:**

- Low: 1 record (1 Yes, 0 No) , H(Low) = 0
- Medium: 2 records (1 Yes, 1 No)

$$H(Medium) = -(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}) = 1$$

- High: 2 records (0 Yes, 2 No) , H(High) = 0
- Weighted Entropy For Income:

$$H(S_{Income}) = \frac{1}{5}.0 + \frac{2}{5}.1 + \frac{2}{5}.0 = 0.4$$

- Information Gain for Income:

IG(Young, Income) = $H(Young)$ - H(S Income) = 0.971 – 0.4 = 0.571

2. Student

- **Split the dataset into subsets:**

- Yes: 2 records (2 Yes, 0 No) , H(Yes) = 0
- No: 3 records (0 Yes, 3 No) , H(No) = 0
- Weighted Entropy For Student:

$$H(S_{Student}) = \frac{2}{5}.0 + \frac{3}{5}.0 = 0$$

- Information Gain for Student :

IG(Young, Student) = $H(Young)$ - H(S Student) = 0.971 – 0 = 0.971

3. Credit Rating

- **Split the dataset into subsets:**

- Fair: 3 records (1 Yes, 2 No)

$$H(\text{Fair}) = -\left(\frac{1}{3}\log_2\frac{1}{3} + \frac{2}{3}\log_2\frac{2}{3}\right) \approx 0.918$$

- Excellent: 2 records (1 Yes, 1 No)

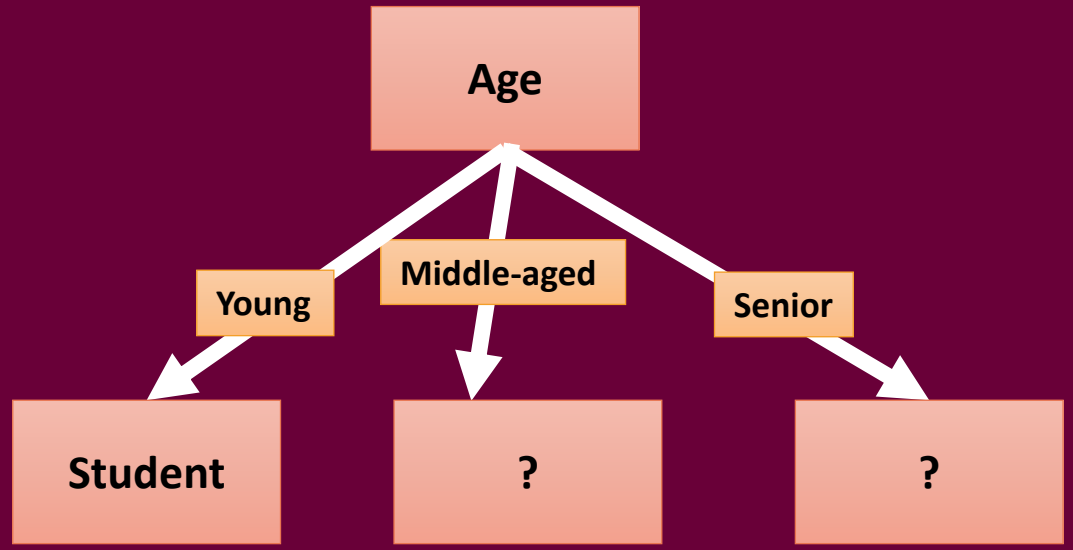
$$H(\text{Excellent}) = -\left(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}\right) = 1$$

- Weighted Entropy For Credit Rating:

$$H(S_{\text{Credit Rating}}) = \frac{3}{5}.0.918 + \frac{2}{5}.1 \approx 0.951$$

- Information Gain for Credit Rating :

IG(Young, Credit Rating) = $H(Young)$ - H(S Credit Rating) = 0.971 – 0.951 = 0.02



Based on the calculated Information Gains:

Income: 0.571	Student: 0.971	Credit Rating: 0.02

The attribute **Student** has the highest Information Gain and is chosen as the root node of the Young Branch.

Middle Edged Branch

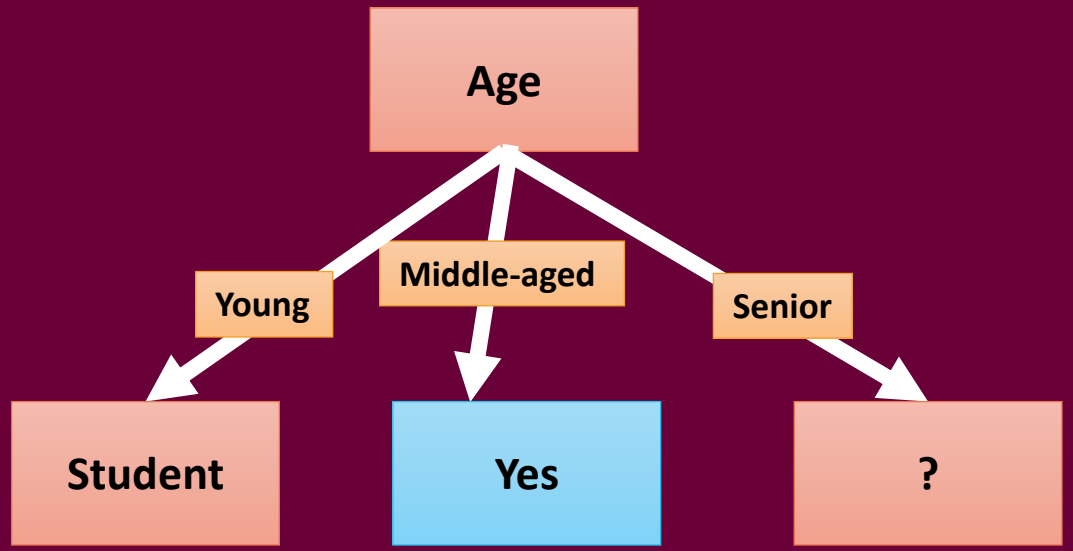
Age	Income	Credit Rating	Buy
Middle-aged	High	Fair	Yes
Middle-aged	Low	Excellent	Yes
Middle-aged	Medium	Excellent	Yes
Middle-aged	High	Fair	Yes

Number of Records = 4

Number of:

Buy = Yes → 4

Buy = No → 0



Calculating the Entropy

$$H(Middle - aged) = 0$$

Senior Branch

Age	Income	Credit Rating	Buy
Senior	Medium	Fair	Yes
Senior	Low	Fair	Yes
Senior	Low	Excellent	No
Senior	Medium	Fair	Yes
Senior	Medium	Excellent	No

Number of Records = 5

Number of:

Buy = Yes → 3

Buy = No → 2

Calculating the Entropy

$$H(Senior) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right)$$

$$H(Senior) = -(0.6\,log2\,0.6 + 0.4\,log2\,0.4) \approx 0.971$$

1. Income

- **Split the dataset into subsets:**

- Low: 2 records (1 Yes, 1 No)

$$H(Low) = -(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}) = 1$$

- Medium: 3 records (2 Yes, 1 No)

$$H(Medium) = -(\frac{2}{3}\log_2\frac{2}{3} + \frac{1}{3}\log_2\frac{1}{3}) \approx 0.918$$

- Weighted Entropy For Income:

$$H(S_{Income}) = \frac{2}{5}.1 + \frac{3}{5}.0.918 = 0.951$$

- Information Gain for Income:

IG(Senior, Income) = H(Senior) - H(S Income) = 0.971 – 0.951 = 0.02

2. Credit Rating

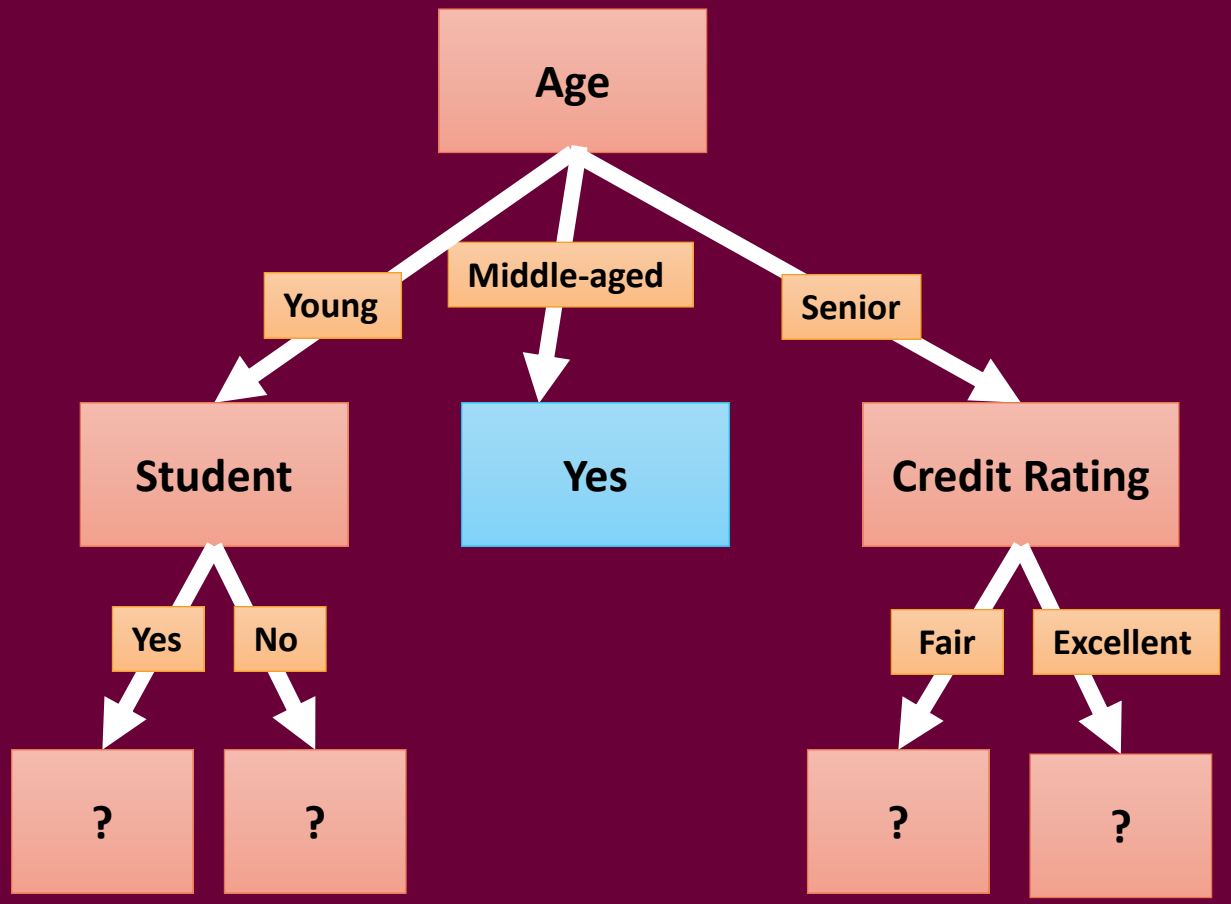
- **Split the dataset into subsets:**

- Fair: 3 records (3 Yes, 0 No), H(Fair) = 0
- Excellent: 2 records (0 Yes, 2 No), H(Excellent) = 0
- Weighted Entropy For Credit Rating:

$$H(S_{Credit\ Rating}) = \frac{3}{5}.0 + \frac{2}{5}.0 = 0$$

- Information Gain for Credit Rating :

IG(Senior, Credit Rating) = H(Senior) - H(S Credit Rating) = 0.971 – 0 = 0.971



Based on the calculated Information Gains:

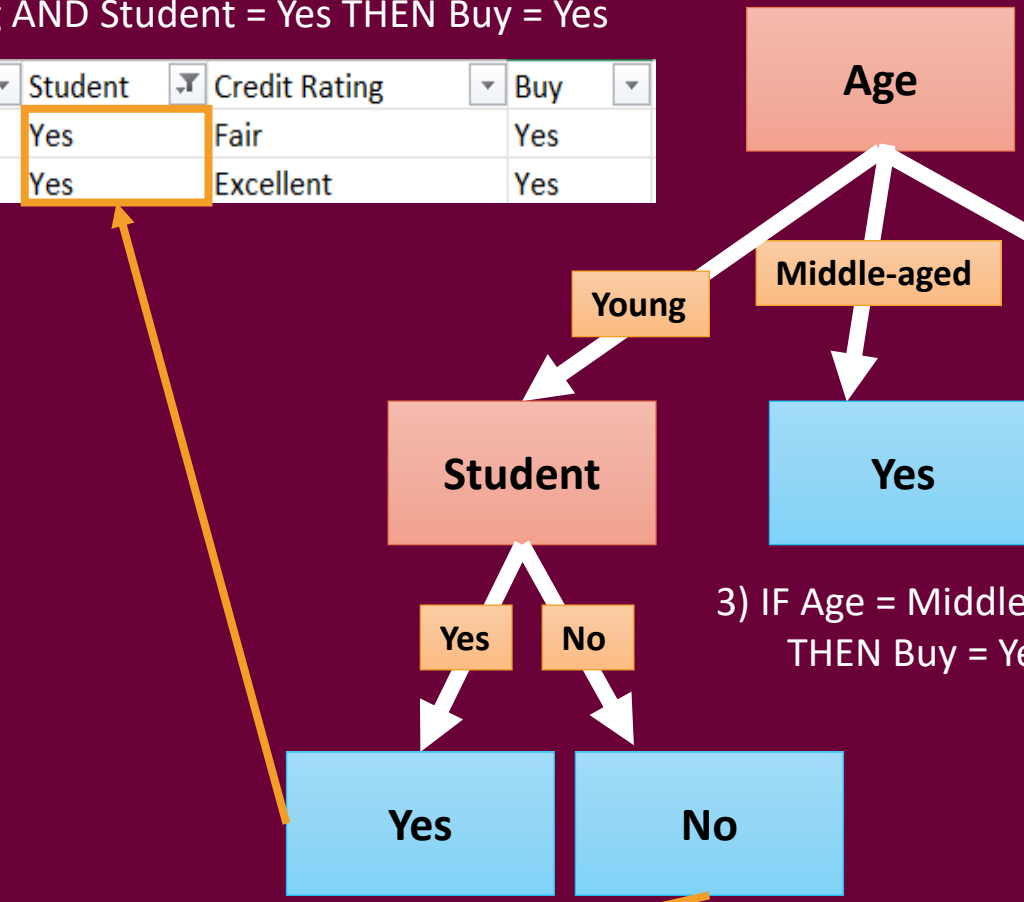
Income: 0.02	Credit Rating: 0.971

The attribute **Credit Rating** has the highest Information Gain and is chosen as the root node of the Senior Branch.

Decision Rules

1) IF Age = Young AND Student = Yes THEN Buy = Yes

Age	Income	Student	Credit Rating	Buy
Young	Low	Yes	Fair	Yes
Young	Medium	Yes	Excellent	Yes



3) IF Age = Middle-aged THEN Buy = Yes

4) IF Age = Senior AND Credit Rating = Fair THEN Buy = Yes

Age	Income	Student	Credit Rating	Buy
Senior	Low	Yes	Excellent	No
Senior	Medium	No	Excellent	No

Age	Income	Student	Credit Rating	Buy
Young	High	No	Fair	No
Young	High	No	Excellent	No
Young	Medium	No	Fair	No

2) IF Age = Young AND Student = No THEN Buy = No

Age	Income	Student	Credit Rating	Buy
Senior	Medium	No	Fair	Yes
Senior	Low	Yes	Fair	Yes
Senior	Medium	Yes	Fair	Yes

5) IF Age = Senior AND Credit Rating = Excellent THEN Buy = No

Gini Index

- When a Decision Tree chooses a feature to split data, it looks for the purest split. This means we want the node to be: **All Yes, or All No, not a mix**.
- That's why we use an **impurity measure**. The two most common measures are: **Gini Index, Entropy**.

$$Gini = 1 - \sum p_i^2$$

Example If:
5 Yes
5 No
Total = 10
Then
p(Yes) = 5/10 = 0.5
p(No) = 5/10 = 0.5
Gini = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5

If a pure node:
10 Yes
0 No
p(Yes) = 10/10 = 1
p(No) = 0/10 = 0
Gini = 1 - (1^2 + 0) = 0

$$Gini\ Gain = Gini(parent) - \sum \frac{n_i}{n} \times Gini(child_i)$$

Iris Binary Classification Using Decision Tree

```
# Train-test Split
from sklearn.model_selection import train_test_split
X = df[df.columns[:4]] # features == first four columns
y = df[df.columns[-1]] # Species == target
X_train,X_test,y_train,y_test =train_test_split(X,y,test_size=0.20) # training = 80 , testing = 20
X_train
```

	Sepal length	Sepal width	Petal length	Petal width
137	6.4	3.1	5.5	1.8
143	6.8	3.2	5.9	2.3
63	6.1	2.9	4.7	1.4
80	5.5	2.4	3.8	1.1
148	6.2	3.4	5.4	2.3
...	...	...	...	...
139	6.9	3.1	5.4	2.1
108	6.7	2.5	5.8	1.8
145	6.7	3.0	5.2	2.3
70	5.9	3.2	4.8	1.8
138	6.0	3.0	4.8	1.8

80 rows × 4 columns

```
# Train Model
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier()
classifier.fit(X_train,y_train) # Model learns from training data
```

```
# Test Model
y_pred=classifier.predict(X_test) # Model predicts the values of the test data.
```

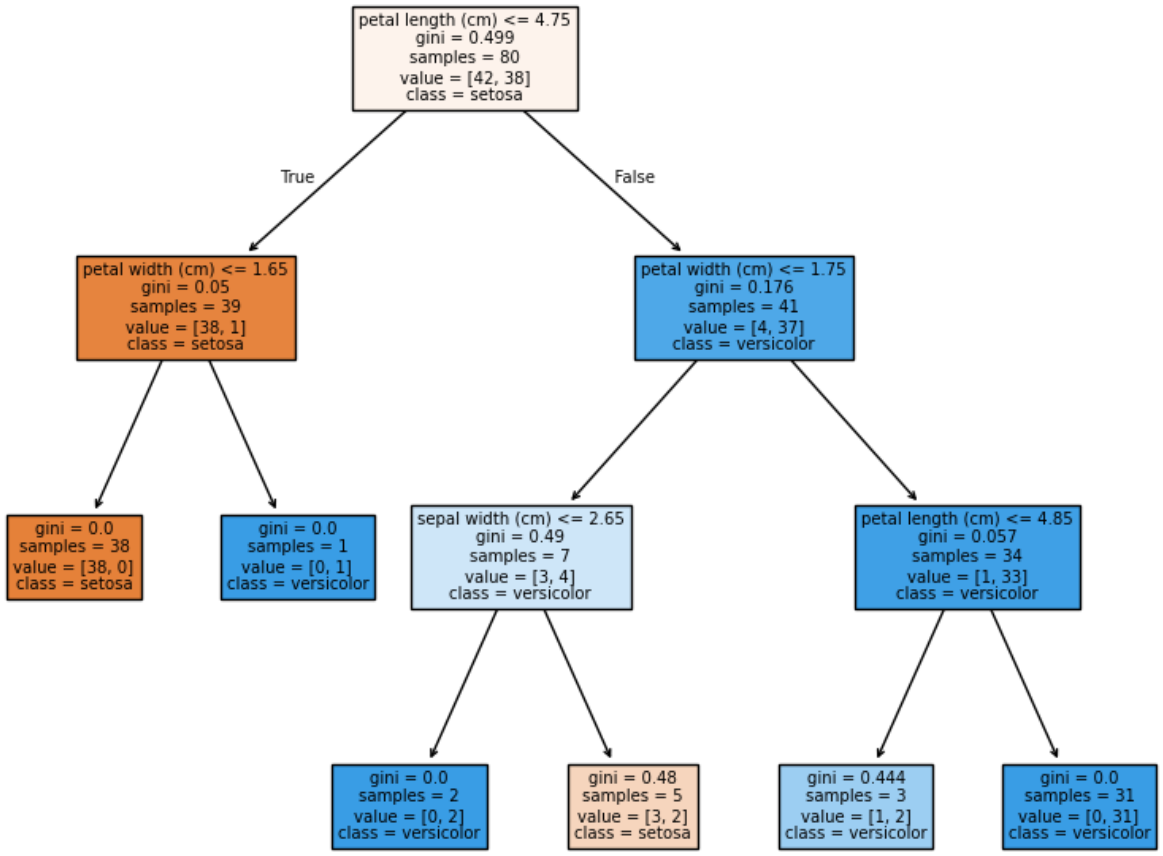
```
# Evaluate Model
from sklearn.metrics import classification_report,confusion_matrix,accuracy_score
print(confusion_matrix(y_test,y_pred))
print('Accuracy:',accuracy_score(y_test,y_pred))
```

```
[[ 7  1]
 [ 0 12]]
Accuracy: 0.95
```

```python
# Visualize the Result
from sklearn import tree
text_representation=tree.export_text(classifier)
print(text_representation) # turns tree into text.
```

```
|--- feature_2 <= 4.75
|   |--- feature_3 <= 1.65
|   |   |--- class: 1
|   |--- feature_3 >  1.65
|   |   |--- class: 2
|--- feature_2 >  4.75
|   |--- feature_3 <= 1.75
|   |   |--- feature_2 <= 5.35
|   |   |   |--- feature_3 <= 1.55
|   |   |   |   |--- feature_0 <= 6.55
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- feature_0 >  6.55
|   |   |   |   |   |--- class: 1
|   |   |   |--- feature_3 >  1.55
|   |   |   |   |--- class: 1
|   |   |--- feature_2 >  5.35
|   |   |   |--- class: 2
|   |--- feature_3 >  1.75
|   |   |--- feature_2 <= 4.85
|   |   |   |--- feature_0 <= 5.95
|   |   |   |   |--- class: 1
|   |   |   |--- feature_0 >  5.95
|   |   |   |   |--- class: 2
|   |   |--- feature_2 >  4.85
|   |   |   |--- class: 2
```

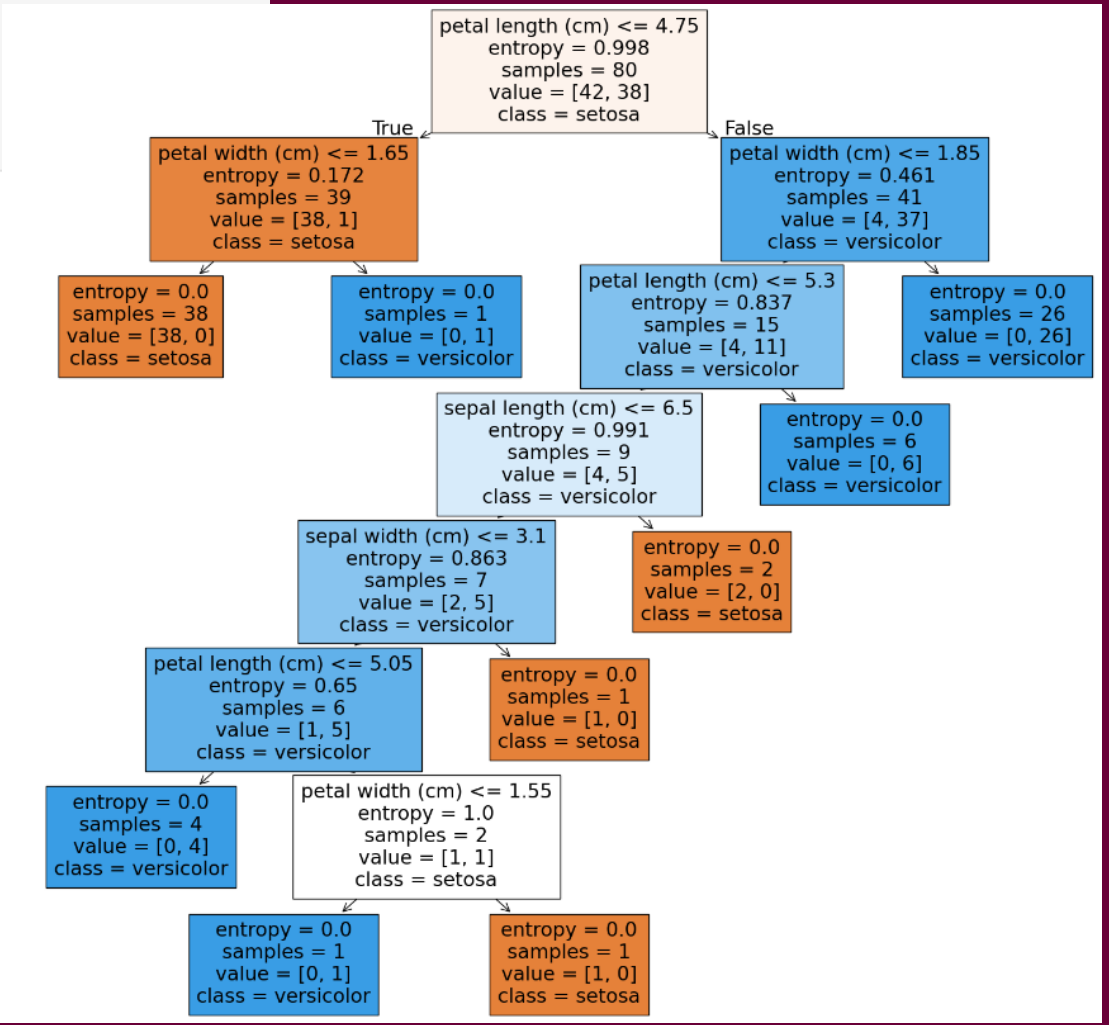
```python
# Visually plots the tree
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(10,8))
_ = tree.plot_tree(classifier, feature_names = iris.feature_names,class_names = iris.target_names, filled=True)

# 1) Condition
# 2) gini = a measure of Node purity.
# 3) samples = number of samples that reached this node during training. (80)
# 4) value = number of samples from each class. [50 class 1, 30 class 2]
# 5) class = final prediction for the node. The tree selects the most common class. The maximum = 50 --> class = class 1
# As we scroll down: samples decrease, gini decreases, until we reach a leaf node.
# Leaf Node: The last node in the tree will be: gini = 0, samples = 38, value = [38,0], class = 1 (All samples are in the same class).
```



Tune the Model (Change hyperparameters, such as criterion, max_depth)

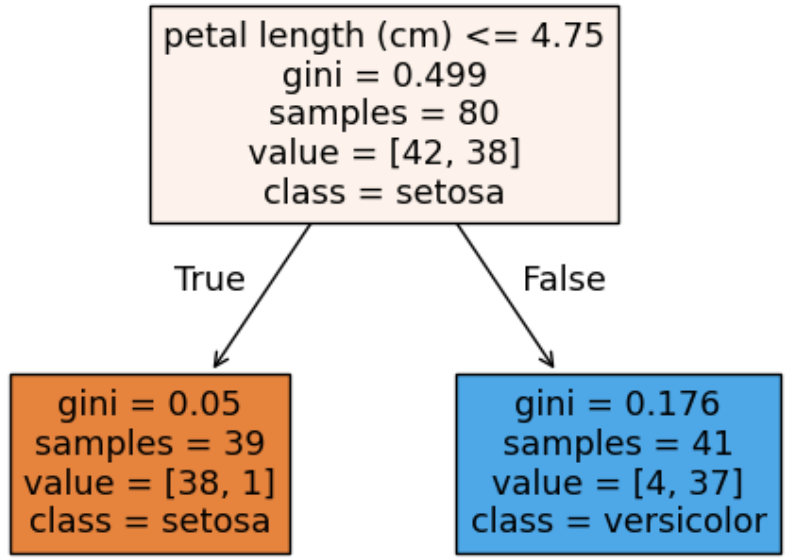
```
# Change criterion = Instead of using Gini Uses Entropy
classifier = DecisionTreeClassifier(criterion= 'entropy')
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test,y_pred))

text_representation = tree.export_text(classifier)
print(text_representation)

fig = plt.figure(figsize=(20,18))
_ = tree.plot_tree(classifier, feature_names=iris.feature_names, class_names=iris.target_names,filled=True)
```

```
[[ 7  1]
 [ 1 11]]
Accuracy: 0.9
|--- feature_2 <= 4.75
|   |--- feature_3 <= 1.65
|   |   |--- class: 1
|   |--- feature_3 >  1.65
|   |   |--- class: 2
|--- feature_2 >  4.75
|   |--- feature_3 <= 1.85
|   |   |--- feature_2 <= 5.30
|   |   |   |--- feature_0 <= 6.50
|   |   |   |   |--- feature_1 <= 3.10
|   |   |   |   |   |--- feature_2 <= 5.05
|   |   |   |   |   |   |--- class: 2
|   |   |   |   |   |--- feature_2 >  5.05
|   |   |   |   |   |   |--- feature_3 <= 1.55
|   |   |   |   |   |   |   |--- class: 2
|   |   |   |   |   |   |--- feature_3 >  1.55
|   |   |   |   |   |   |   |--- class: 1
|   |   |   |   |--- feature_1 >  3.10
|   |   |   |   |   |--- class: 1
|   |   |   |--- feature_0 >  6.50
|   |   |   |   |--- class: 1
|   |   |--- feature_2 >  5.30
|   |   |   |--- class: 2
|   |--- feature_3 >  1.85
|   |   |--- class: 2
```



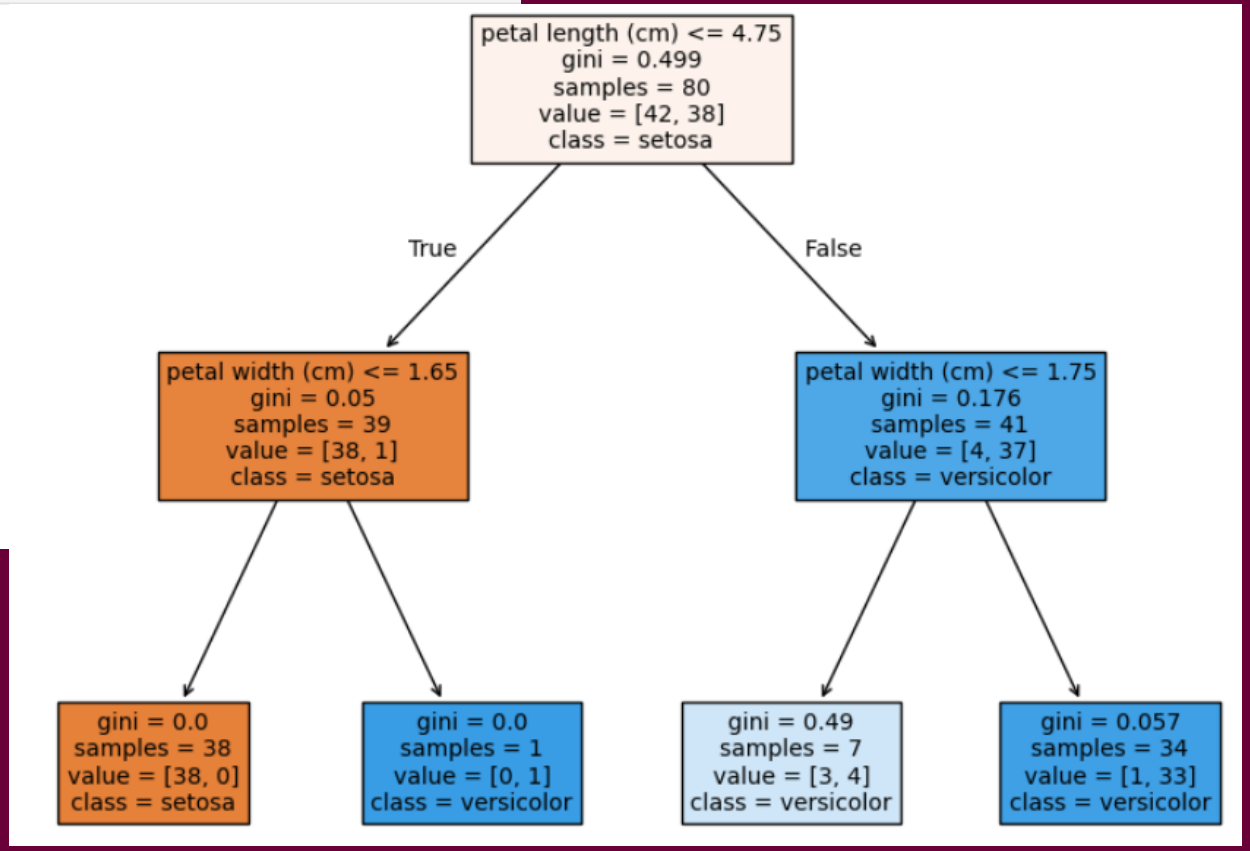
```python
# Max Depth: The maximum number of levels (Levels) that a tree is allowed to reach from the root to the leaf.
classifier = DecisionTreeClassifier(max_depth=1) # tree will have only one level.
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test,y_pred))

text_representation = tree.export_text(classifier)
print(text_representation)

fig = plt.figure()
_ = tree.plot_tree(classifier,feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
```

```
[[ 6  2]
 [ 0 12]]
Accuracy: 0.9
|--- feature_2 <= 4.75
|   |--- class: 1
|--- feature_2 >  4.75
|   |--- class: 2
```



```python
classifier = DecisionTreeClassifier(max_depth=2)
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test,y_pred))

text_representation = tree.export_text(classifier)
print(text_representation)

fig = plt.figure(figsize=(10,8))
_ = tree.plot_tree(classifier, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
```

```
[[ 6  2]
 [ 0 12]]
Accuracy: 0.9
|--- feature_2 <= 4.75
|   |--- feature_3 <= 1.65
|   |   |--- class: 1
|   |--- feature_3 >  1.65
|   |   |--- class: 2
|--- feature_2 >  4.75
|   |--- feature_3 <= 1.75
|   |   |--- class: 2
|   |--- feature_3 >  1.75
|   |   |--- class: 2
```



```python
classifier = DecisionTreeClassifier(max_depth=3)
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test,y_pred))

text_representation = tree.export_text(classifier)
print(text_representation)
```

```
[[ 8  0]
 [ 0 12]]
Accuracy: 1.0
|--- feature_2 <= 4.75
|   |--- feature_3 <= 1.65
|   |   |--- class: 1
|   |--- feature_3 >  1.65
|   |   |--- class: 2
|--- feature_2 >  4.75
|   |--- feature_3 <= 1.75
|   |   |--- feature_2 <= 5.35
|   |   |   |--- class: 1
|   |   |--- feature_2 >  5.35
|   |   |   |--- class: 2
|   |--- feature_3 >  1.75
|   |   |--- feature_2 <= 4.85
|   |   |   |--- class: 2
|   |   |--- feature_2 >  4.85
|   |   |   |--- class: 2
```

Testing the Optimal Tree Depth

```python
def tree_depth_tuning(d):
    classifier = DecisionTreeClassifier(max_depth=d)
    classifier.fit(X_train, y_train)

    y_pred = classifier.predict(X_test)
    accuracy = accuracy_score(y_test,y_pred)
    return accuracy
```

```python
tree_results = pd.DataFrame({'D':np.arange(1, 10)})
```

```python
tree_results['Accuracy'] = tree_results['D'].apply(tree_depth_tuning)
tree_results
```

	D	Accuracy
0	1	0.90
1	2	0.90
2	3	1.00
3	4	0.95
4	5	0.95
5	6	0.95
6	7	0.95
7	8	0.90
8	9	0.95

Small Depth == Underfitting (Not enough)

Appropriate Depth == Best Performance (Enough)

Large Depth == Overfitting (The model saves the answers)

Task

- Classification of the three flower types in Iris Dataset (Setosa, Versicolor, Virginica) using KNN.
- Classification of the three flower types in Iris Dataset (Setosa, Versicolor, Virginica) using RNN.
- Classification of the three flower types in Iris Dataset (Setosa, Versicolor, Virginica) using DECISION TREE.

Thank You